

Windows PC に XAMPP をインストールする手順です。

****STEP 1：インストーラーのダウンロード****

1. ブラウザを開き、XAMPP の公式サイト ([XAMPP Installers and Downloads for Apache Friends](#)) にアクセスします。
2. 画面にある **「Windows 向け XAMPP」** という大きなボタンをクリックします。
3. ダウンロードが自動的に始まります (数分かかる場合があります)。

****STEP 2：インストールの実行****

1. ダウンロードしたファイル (xampp-windows-x64-x.x.x-installer.exe) をダブルクリックして開きます。
2. ****警告メッセージが表示された場合****:
 - * 「ユーザーアカウント制御 (UAC)」の警告画面が出たら **「はい」** を押します。
 - * 「Warning: Important Question...」という英字の警告が出た場合は **「OK」** を押して進めます。
3. セットアップ画面 (Setup - XAMPP) が開いたら、以下の順でボタンを押して進めます。
 - **Welcome to the XAMPP****: **「Next」** をクリック。
 - **Select Components****: コンポーネント選択画面です。初期状態 (すべてにチェックが入った状態) のまま **「Next」** をクリック。
 - **Installation folder****: インストール先のフォルダ選択です。初期設定の **「C:\xampp」** のまま **「Next」** をクリック。
 - **Language****: 言語選択です。「English」のまま **「Next」** をクリック。
 - **Ready to Install****: **「Next」** を押すと実際のインストールが始まります (3~5 分ほど待ちます)。

****STEP 3：インストール完了と初回起動****

1. 画面に「Completing the XAMPP Setup Wizard」と表示されたらインストール完了です。
 2. **「Do you want to start the Control Panel now?」** (コントロールパネルを起動しますか?) にチェックが入っていることを確認し、**「Finish」** を押します。
 3. XAMPP コントロールパネルが起動したら、**「Apache」** と **「MySQL」** の右側にある **「Start」** ボタンをそれぞれ押します。
- > ****※セキュリティの警告が出た場合****
- > 「Windows ファイアウォールによってこのアプリの機能のいくつかがブロックされています」という画面が出たら、**「アクセスを許可する」** (または「プライベート ネットワーク」にチェックを入れて許可) をクリックしてください。
- 緑色の背景で **「Running」** と表示されれば、環境構築は完了です。

🚀 【旅行プラン作成・共有ツール】開発スタートマニュアル

チーム開発を始めるための初期設定手順です。順番に進めてみてください！

【事前準備】必要なソフトのインストール

まだ入っていないものがあれば、事前にインストールしておいてください。

1. ****VSCode**** (コードを書くソフト)
2. ****Git**** (プログラムの変更を記録・共有するツール)
3. ****XAMPP**** (Web サーバーとデータベースを動かすソフト)

STEP 1：プロジェクトのコードをパソコンにダウンロードする

1. ****VSCode**** を開きます。
2. 上部メニューの ****「ターミナル」 > 「新しいターミナル」**** を選択します。
3. 以下のコマンドを入力して Enter を押し、`htdocs` フォルダ (Web サイト用の場所) へ移動します。

```
cd C:\xampp\htdocs
```

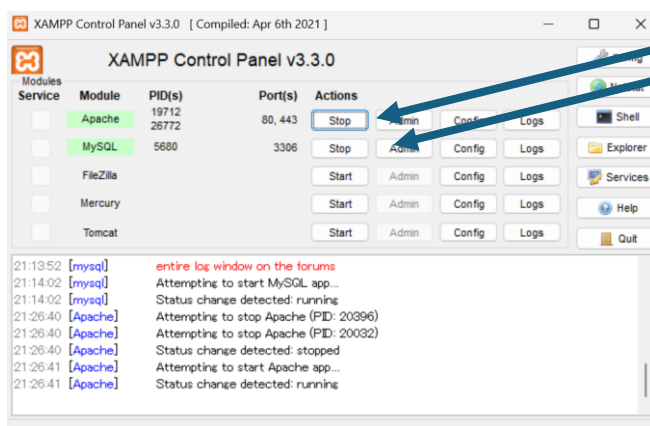
4. 以下のコマンドを実行して、共有リポジトリからコードをダウンロードします。

```
git clone https://github.com/mitsuki0084/TECH-BASE-A7.git
```

5. `htdocs` の中に `TECH-BASE-A7` フォルダが作成されていれば成功です！

STEP 2：XAMPP を起動してデータベースを準備する

1. ****XAMPP コントロールパネル**** を開きます。

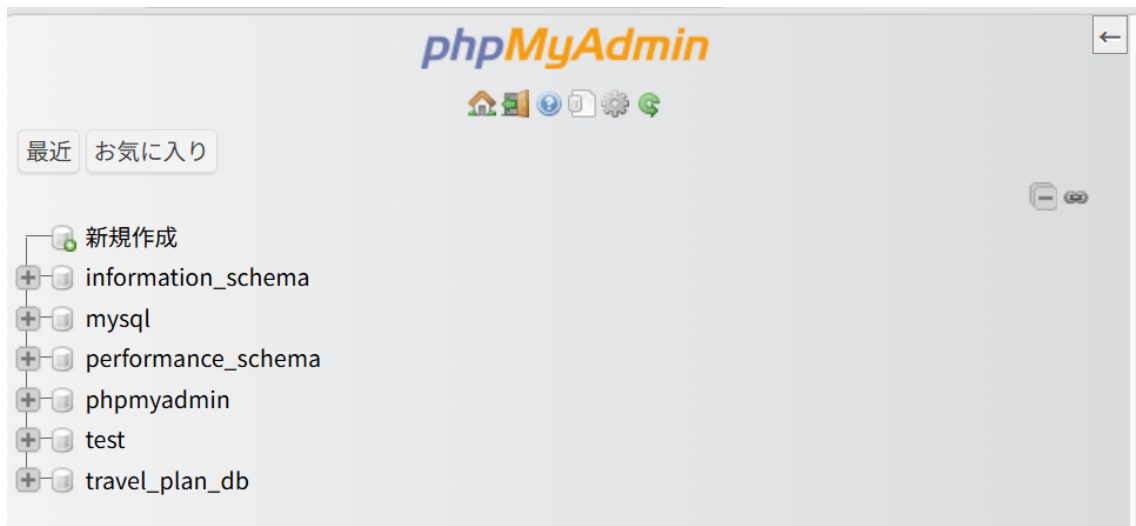


2. **「Apache」** と **「MySQL」** の横にある **「Start」** ボタンをそれぞれ押します。(上の画像のように両方の名前が緑色になれば OK)

3. ブラウザ（Chrome など）を開き、アドレスバーに以下を入力して Enter を押します。

<http://localhost/phpmyadmin/>

以下ようになったときは右上の←を押す



下のようなになったら左上がメニューボタン



💡 補足：phpMyAdmin（データベース管理ツール）の基本操作

<http://localhost/phpmyadmin/> は、データベースの中身を目で見えて操作できる管理画面です。開発中にデータを確認したいときに使います。

● データの中身を見る方法

1. メニュー→データベース→**travel_plan_db** をクリック。
2. 表示されたテーブル（users や plans など）をクリック。
3. 「表示」タブで、登録されているデータの一覧を確認できます。

● 初期状態に戻したい場合（データが崩れたとき）

1. 左側の travel_plan_db を選択 > 上部メニューの「操作」をクリック。
2. 下部にある「データベースを削除する (DROP DATABASE)」を実行。
3. 再度「インポート」から schema.sql を読み込めば初期状態に戻せます。

4. 画面上のメニューから **「インポート」** タブをクリックします。

5. **「ファイルを選択」** を押し、以下のファイルを選択します。

* 場所: `C:\xampp\htdocs\TECH-BASE-A7\sql\schema.sql`

6. ページ一番下の **「実行」** ボタンを押します。

* 画面左側に `travel\_plan\_db` というデータベースが作られれば完了です。

STEP 3：Web サイトが表示されるか確認する

ブラウザのアドレスバーに以下を入力してアクセスします。

<http://localhost/TECH-BASE-A7/index.php>

「旅プランナー」というヘッダー画面が表示されれば、開発環境の準備はすべて完了です！

STEP 4：自分の作業用ブランチを作って開発を始める

全員が同じ場所（main）にコードを直接書くと衝突してしまうため、**自分専用の作業部屋（ブランチ）** を作って開発します。

1. VSCode で `C:\xampp\htdocs\TECH-BASE-A7` フォルダを開きます。
2. ターミナルを開き、ブランチを作成します。以下の例のように feature/のあとを機能名 + 名前にする。

1. みつき（バックエンド / 天気 API 担当）

```
git checkout -b feature/weather-api-mitsuki
```

2. メンバーA（ログイン・会員登録担当）

```
git checkout -b feature/auth-tsumu
```

3. 画面左下の表示が作成したブランチ名（`feature/...`）に変わっていれば準備 OK です！
わからないところがあれば気軽に聞いてください！

📖【初心者向け】チーム開発で使う Git & GitHub 基本ガイド

チームでプログラムを開発するときは、コードの競合（衝突）を防ぐために「作業用ブランチ（自分の作業部屋）」を作って作業し、完成したら「Pull Request（プルリクエスト）」を出して合流させるのが基本ルールです。

1. Git の全体イメージ（用語の解説）

- **main（メインブランチ）**：完成したコードを置く「本番用の部屋」。※ここに直接コードを書かないでください！
- **feature/〇〇（機能ブランチ）**：自分の作業を行う「作業用の部屋」。
- **コミット（Commit）**：作業の区切りで「保存」すること。
- **プッシュ（Push）**：ローカル（自分の PC）で保存したデータを、GitHub（ネット上）へ送信すること。
- **プルリクエスト（Pull Request / PR）**：「自分の作業が終わったので main に合流させてください」とチームに依頼すること。

2. 開発で使う基本コマンド（日々の作業の流れ）

普段の開発は、以下の 4 つのステップを繰り返して進めます。

STEP 1：作業を始める（自分のブランチを作る）

まずは自分専用の作業部屋を作成して移動します。

自分の作業ブランチを作成して切り替える（初回のみ）

`git checkout -b feature/自分の担当機能名+名前`

例: ログイン機能を作る場合

`git checkout -b feature/auth-name`

(VSCode の左下に feature/auth+name が表示されていれば OK です)

STEP 2：作業した内容を保存する（コミット）

コードを少し書いたり修正したら、こまめに保存（コミット）します。

1. 変更した全ファイルを保存対象に追加

`git add .`

2. 変更内容のメモを残して保存（コミット）“この中に変更内容を簡潔に説明して書く”

`git commit -m "ユーザー登録フォームの見た目を作成"`

STEP 3：保存したコードを GitHub に送る（プッシュ）

ローカル（自分の PC）で保存した内容を、ネット上の GitHub へアップロードします。

初めてそのブランチを送信するとき

`git push -u origin feature/自分の担当機能名`

2 回目以降の送信

`git push`

STEP 4：GitHub でプルリクエスト（PR）を出す

1. GitHub のリポジトリページを開きます。
2. 画面上に「**Compare & pull request**」という黄色のボタンが出ているのでクリックします。
3. タイトルと「何を作ったか (例: ログイン画面の HTML と処理を作成)」を入力し、「**Create pull request**」を押します。
4. みつきに「○○のプルリクを出しました!」と報告します。

3. 困ったときのトラブル対処 (Q&A)

Q1. 今自分がどのブランチにいるか分からない!

git branch

(* (星マーク) がついている場所が今いるブランチです)

Q2. すでに存在するブランチに切り替えたい!

git checkout feature/自分の担当機能名

Q3. 自分の PC のコードを最新状態にしたい!

チームの誰かが main を更新したときは、最新のコードを取り込みます。

main ブランチに移動

git checkout main

最新のコードをダウンロード

git pull

Q4. 何かエラーが出てコマンドが動かなくなった!

無理にコマンドを連打せず、画面に表示された赤字のエラーメッセージをそのままコピーしてリーダーに相談してください!

⚠ チーム開発で絶対に守る「4つの約束 (Git ルール)」

1. main ブランチには絶対に直接コミット・プッシュしない

- **理由:** main は「常に正しく動く完成品」を置いておく場所です。全員が直接書き込むと、不具合のあるコードで全体が動かなくなってしまう。
- **ルール:** どんな小さな修正 (1 行の修正など) でも、必ず自分の feature/○○ ブランチを作って作業してください。

2. コミットメッセージ (保存時のメモ) は分かりやすく書く

- **悪い例:** git commit -m "修正" / git commit -m "aaa" (後から何をしたか全く分からない)
- **良い例:** git commit -m "ログイン画面の HTML 作成" / git commit -m "バグ修正: パスワード入力欄のレイアウト崩れ"
- **ポイント:** 「何を追加・変更したか」が一行で分かるように書きましょう。

3. 毎日作業を始める前に main を最新にする

他のメンバーが作った機能が main に追加されている可能性があるため、**作業開始時には必ず最新状態を取り込みます。**

1. main ブランチに移動

```
git checkout main
```

2. 最新のコードをダウンロード

```
git pull origin main
```

3. 自分の作業ブランチに戻る

```
git checkout feature/自分のブランチ名
```

4. 最新の main の内容を自分のブランチに取り込む

```
git merge main
```

これらをしてから作業に入る！

これを習慣にするだけで、後からコードが衝突（コンフリクト）する確率を激減させることができます。

4. データベースの構造（schema.sql）を変えたらすぐに報告する

- **理由:** 誰かがテーブルを追加したり列（カラム）を増やしたりしたのに黙っていると、他の人の PC でエラーが発生します。
- **ルール:** データベースを変更したら、schema.sql を更新してコミットし、チームのチャットで「DB の更新（テーブル追加）をしたので、みんな schema.sql を再インポートしてね！」とアナウンスしてください。

インポート：<http://localhost/phpmyadmin/>で phpmyadmin にアクセスしてインポートする。

Mission7 の機能一覧・担当・進め方まとめ

****1. 作成する機能一覧****

- ***ユーザー認証機能** (会員登録・ログイン・ログアウト)
- ***ユーザー設定機能** (プロフィール確認・編集)
- ***プラン基本機能** (プラン一覧表示・詳細表示・新規投稿・編集・削除)
- ***プラン検索・タグ機能** (キーワード検索・エリアやタグでの絞り込み)
- ***天気情報表示機能** (外部 API と連携した目的地天気表示)
- ***管理者機能** (ユーザー管理・投稿プランの強制削除)

****2. 担当者別機能一覧と実装順序****

****【こま】担当：ログイン・会員登録（基本機能）****

1. ****会員登録機能****: ユーザーが名前・メールアドレス・パスワードを入力して登録する。
2. ****ログイン機能****: 登録情報と照合し、ログイン状態（セッション）を作る。

****【かじもん】担当：ユーザー設定・ログアウト（基本機能）****

1. ****ログアウト機能****: ログイン状態（セッション）を破棄してトップ画面へ戻る。
2. ****ユーザー情報編集機能****: マイページから登録名やメールアドレスを変更・保存する。

****【うめ】担当：一般機能グループ（プラン操作メイン）****

1. ****プラン一覧・詳細画面****: DB からデータを取得して一覧および個別詳細を表示する。
2. ****プラン作成機能****: フォームから旅行のタイトル・目的地・日程を入力して DB に保存する。
3. ****プラン編集・削除機能****: 自分が投稿したプランの内容変更・削除を行う。

****【つむ】担当：一般機能&管理機能グループ（検索・管理者メイン）****

1. ****プラン検索・タグ絞り込み機能****: 目的地やタグ（「関東」「温泉」など）で条件検索する。
2. ****管理者用ダッシュボード****: 管理者だけがアクセスできる専用ページを作成する。
3. ****管理用削除機能****: 不適切な投稿プランや違反ユーザーを管理者権限で削除する。

****【みつき】担当：API 連携・全体統合****

1. ****天気 API 連携****: 目的地の情報をもとに外部天気 API からデータを取得し、詳細画面へ非同期で表示させる。
2. ****コードレビュー・結合****: メンバーから出されたプルリクエスト（PR）を確認し、`main` へ合流させる。
3. ****全体調整・バグ修正****: 全体の動作テストを行い、エラーが発生した箇所のサポート・修正を行う。

****3. 各プログラムの仕様説明****

*****ユーザー認証・設定（担当：こま / かじもん）****

- * `register.php` / `login.php`: フォーム送信されたパスワードを `password\_hash()` で安全に暗号化して DB（`users` テーブル）に保存・照合します。ログイン成功時は `\$\_SESSION['user\_id']` に情報を保持します。
- * `logout.php`: `session\_destroy()` を実行してログイン状態を解除します。

*****プラン基本機能（担当：うめ）****

- * `plan\_list.php` / `plan\_detail.php`: DB から `SELECT` 文でデータを取り出し HTML に埋め込みます。
- * `plan\_create.php` / `plan\_edit.php` / `plan\_delete.php`: フォームから受け取ったデータを `INSERT` / `UPDATE` / `DELETE` 文で DB へ反映します。

*****検索・管理機能（担当：つむ）****

- * `search.php`: 検索窓の文字をもとに SQL の `LIKE '%キーワード%'` や `WHERE tag\_id = ...` を組み立ててデータを抽出します。
- * `admin.php`: ログイン中のユーザーが管理者（`is\_admin = 1`）かどうかを判定し、管理者以外はアクセスを拒否（リダイレクト）する保護をかけます。

*****天気 API 連携（担当：みつき）****

- * `weather.js`（または PHP 側）: 目的地名（例: 「Tokyo」）をパラメーターとして外部 API へ送信し、返ってきた JSON データ（天気・気温）を画面の指定要素（`#weather-info`）に表示します。

****4. 作業時に現状をブラウザで確認する方法****

1. ****XAMPP の起動確認****: 「Apache」と「MySQL」が起動（緑色）になっているか確認します。
2. ****ブラウザでアクセス****: アドレスバーに以下を入力して開きます。

<http://localhost/TECH-BASE-A7/index.php>

3. ****コード変更が反映されない場合****:

* ブラウザが古い情報をキャッシュしている可能性があるため、****`Ctrl` + `F5`**** (Mac は ****`Cmd` + `Shift` + `R`****) を押して強制再読み込み (ハードリフレッシュ) を行ってください。

****5. 機能がうまくいかず 1 個前のコミットに戻りたいときの方法****

作業中、コードがぐちゃぐちゃになって「保存する前 (直前のコミット状態)」に戻りたい場合の手順です。

****ケース A : まだ `git commit` していない作業をすべて捨てる場合 (一番よく使う) ****最後にコミットした状態まで一発でコードを巻き戻します。

`git restore .`

****ケース B : 誤って `git commit` までしてしまい、1 つ前のコミットを取り消したい場合****

直前のコミットを取り消し、コードも 1 つ前の状態に戻します。

`git reset --hard HEAD~1`

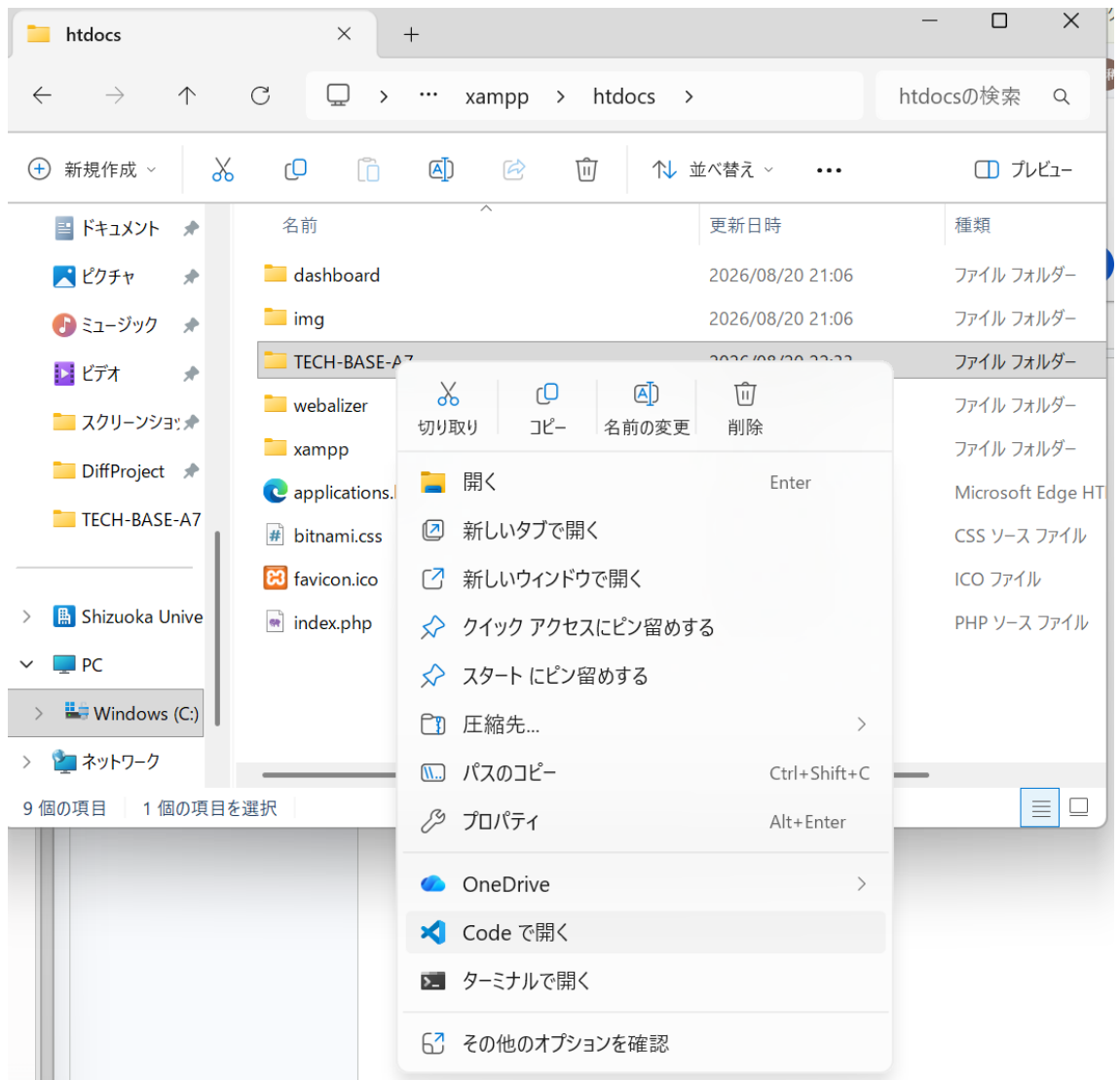
***※注意:** どちらのコマンドも、戻すと書いたコードが消えて復元できなくなります。困ったときや不安なときは、自分でコマンドを実行せずすぐに画面を見せて相談してください。*

作業の手順

エクスプローラーで C:\xampp\htdocs\TECH-BASE-7 を開く

以下のように TECH-BASE-7 のフォルダを右クリックして Code で開くを選択。

※Code で開くが出てこない場合は、スタートメニューから VSCode を開いてメニュー→フォルダを開く→TECH-BASE-7 を選択して開く



自分の担当するプログラムを選択して実装を進める。実装を進める都度 Ctrl+S で保存してね。保存しないと変更が適用されないよ。

自分の実装した内容をブラウザで表示確認したいときは以下の手順をするよ。

XAMPP の起動確認: 「Apache」と「MySQL」が起動（緑色）になっているか確認します。

ブラウザでアクセスアドレスバーに以下を入力して開きます。

<http://localhost/TECH-BASE-A7/index.php>

作業が進んだら Git でコミットしてね。最低でもその日の作業終わるときに 1 度はコミットしてね。コミットしておけばいつでもその時のプログラムを見れるよ。機能が完成したらみんながその機能を使えるように main に反映するからプルリクエストしてね。詳細は上の方に Git の使い方があるよ。

以下共有すべき事項

チーム統一ルール（共通変数・DB・キー名）

全員のプログラムを結合した際にエラーが出ないように、以下の名前を**そのまま共通で使用**してください。

セッション変数名（ログイン状態の保持）

`\$\_SESSION['user\_id']` : ログイン中のユーザーID (数値)

`\$\_SESSION['user\_name']` : ログイン中のユーザー表示名

`\$\_SESSION['is\_admin']` : 管理者フラグ (`0` : 一般ユーザー, `1` : 管理者)

データベーステーブルとカラム名

`users` テーブル: `id`, `name`, `email`, `password`, `is\_admin`

`plans` テーブル: `id`, `user\_id`, `title`, `destination`, `start\_date`, `end\_date`, `tag\_id`, `description`, `created\_at`

`tags` テーブル: `id`, `name`

URL パラメーター（画面遷移時の ID 渡し）

プラン詳細・編集・削除時: `?id=1` (例: `plan\_detail.php?id=1`)

検索時: `?keyword=東京&tag\_id=2` (例: `search.php?keyword=東京&tag\_id=2`)

担当者別の詳細仕様と必要機能

【こま】 担当：ログイン・会員登録

`register.php` (会員登録機能)

必要な要素: フォーム (名前 `name` / メール `email` / パスワード `password`)。

必須処理:

1. 送信されたパスワードを `password\_hash(\$\_POST['password'], PASSWORD\_DEFAULT)` で暗号化する。
2. `users` テーブルへ `INSERT` 実行。
3. 成功したら `login.php` へ自動遷移 (`header('Location: login.php');`)。

`login.php` (ログイン機能)

必要な要素: フォーム (メール `email` / パスワード `password`)。

必須処理:

1. メールアドレスで DB (`users`) を検索。
2. `password\_verify(\$\_POST['password'], \$user['password'])` でパスワード照合。
3. 一致したら `\$\_SESSION['user\_id']`, `\$\_SESSION['user\_name']`, `\$\_SESSION['is\_admin']` に値をセット。
4. `index.php` へ遷移。

【かじもん】 担当：ユーザー設定・ログアウト

`logout.php` (ログアウト処理)

必須処理: `session\_start()` 実行後、`\$\_SESSION = []` で全消去し、`session\_destroy()` を実行。`index.php` へ遷移。

`mypage.php` / `user\_edit.php` (マイページ・会員情報編集)

アクセス制限: `if (empty(\$\_SESSION['user\_id']))` の場合は `login.php` へ強制遷移。

必須処理:

1. `SELECT FROM users WHERE id = \$\_SESSION['user\_id']` で自分の情報を取得してフォームに表示。
2. 編集後、`UPDATE users SET name = ..., email = ... WHERE id = \$\_SESSION['user\_id']` を実行。

【うめ】 担当：プラン基本機能 (作成・一覧・詳細・編集・削除)

`plan\_list.php` (一覧表示) / `plan\_detail.php` (詳細表示)

必須処理: `SELECT plans., users.name FROM plans JOIN users ON plans.user\_id = users.id` を実行し、`foreach` で画面に表示。

API 連携用の必須記述 (重要): `plan\_detail.php` の目的地を表示する HTML 要素には、`id="destination-name"` を必ず付与してください。(例: `

`plan\_create.php` (プラン作成)

必須処理: 未ログイン時はリダイレクト。フォームからデータを受け取り、`user\_id` には `\$\_SESSION['user\_id']` の値をセットして `plans` テーブルへ `INSERT`。

`plan\_edit.php` / `plan\_delete.php` (編集・削除)

セキュリティ処理: 他人の投稿を勝手に消せないよう、SQL には必ず `WHERE id = \$plan\_id AND user\_id = \$\_SESSION['user\_id']` の条件を入れる。

【つむ】担当: プラン検索・管理者機能

`search.php` (検索・絞り込み)

必須処理: `GET` で渡された `keyword` と `tag\_id` を取得。

SQL 組み立て:

キーワードあり: `WHERE title LIKE '%キーワード%' OR destination LIKE '%キーワード%'`

タグ指定あり: `WHERE tag\_id = タグ ID`

`admin.php` (管理者ダッシュボード&削除)

権限チェック: `if (empty(\$\_SESSION['is\_admin']) || \$\_SESSION['is\_admin'] != 1)` の場合はアクセス拒否してトップページへリダイレクト。

必須処理: 全ユーザー・全プランの一覧表示。各行に「強制削除」ボタンを置き、管理者権限で `DELETE FROM plans WHERE id = \$id` を実行できる処理を作成。

【みつき】担当: 天気 API 連携・全体結合

`weather.js` (天気 API 取得処理)

必須処理:

1. JavaScript で `document.getElementById('destination-name').textContent` を読

み取る。

2. 取得した目的地名を使って Weather API へリクエストを送信。

3. 返ってきた天気（晴れ・雨等）・気温データを `plan\_detail.php` 内の `id="weather-info"` 要素へ挿入。

全体統合:

チームメンバーからの PR（プルリクエスト）コードに、上記の命名ルール・セッション処理が正しく使われているか確認してマージ。